ICE Production Deployment Plan

ICE Production Deployment Plan

> **Goal:** Deploy the Interactive Curriculum Engine (ICE) to a publicly accessible URL for $0/month (proof-of-concept / demo), with a clear fallback if free tiers are insufficient.
> **Status:** Plan. Execute the checklist in §10.

Table of Contents

---

1. Architecture Overview

1.1 Final Decisions

Decision	Choice
Budget	Strictly $0 (free tiers / student credits)
Compute VM	**Oracle Cloud Always Free (Ampere A1 ARM, 24 GB RAM)** primary, Azure for Students fallback
Topology	Co-located: API + Worker + Caddy on a single VM via `docker-compose.prod.yml`
Frontend	**Vercel** (native Next.js hosting)
Postgres	**Neon** (0.5 GB free, never expires, pgvector supported)
Redis	**Upstash** (10K cmds/day free; provision 3 free DBs)
Object storage	**Cloudflare R2** (10 GB free, zero egress)
Judge0	**Disabled** — `SANDBOX_BACKEND=subprocess` (Python-only sandbox on API host)
ASR/OCR	**CPU-only** (`ASR_MODEL=tiny` , `OCR_GPU_ENABLED=false`)
CI/CD	**GitHub Actions** (extend existing `deploy.yml`)
Keep-alive	**UptimeRobot** pinging `/api/health` every 5 min
TLS/domain	`<VM_IP>.sslip.io` (Caddy auto-issues Let's Encrypt certs)

1.2 Target Topology

1.3 Component Responsibility Matrix

Component	Hosted on	Why
Next.js frontend	Vercel	Native Next.js support, auto-build, global CDN, no spin-down, generous free tier
FastAPI (`ice-api`)	Oracle A1 VM (Caddy → uvicorn)	Needs to call internal worker; shares filesystem-less env with worker via env vars
Celery worker (`ice-worker`)	Oracle A1 VM	Needs ~1-2 GB RAM for Whisper + Playwright + Remotion + ffmpeg; **must not spin down** (kills queue polling)
Caddy (reverse proxy + TLS)	Oracle A1 VM	Auto Let's Encrypt, simple Caddyfile, ARM image available
PostgreSQL 16 + pgvector	Neon (serverless)	Real managed Postgres with pgvector; free tier never expires (unlike Render's 30-day expiry)
Redis (broker + backend + cache)	Upstash (serverless)	TLS-native, 10 free DBs, 10K cmds/day
Object storage (videos/audio/transcripts)	Cloudflare R2	S3-compatible, 10 GB free, **zero egress** (critical for video streaming)
Image registry	GHCR	Already wired in `deploy.yml` ; free for public repos
Uptime monitor	UptimeRobot	50 free monitors, 5-min interval, doubles as keep-alive

---

2. Platform Selection Rationale

2.1 Why not Render free tier?

Render's free tier is the most developer-friendly PaaS but has three hard limits that disqualify it for ICE:

1. **Free Postgres expires after 30 days** (then a 14-day grace period → deletion). This is a non-starter for any deployment you want to keep alive.
2. **Free web services spin down after 15 min of no inbound traffic.** A Celery worker is long-running and polls Redis for work — it has no inbound HTTP. When it spins down, queued pipelines stall silently until a manual nudge.
3. **Free instances are 512 MB RAM / 0.1 CPU.** ICE's worker image contains ffmpeg + Playwright/Chromium + Node 22 + Remotion + 13 AI libraries. At idle this is ~1 GB; under a Whisper + OCR pipeline it peaks at ~2-3 GB. It will OOM on 512 MB.
4. **Free web services can't send SMTP** (ports 25/465/587 blocked). Email verification and the support-ticket flow would break.

2.2 Why not Fly.io?

Fly.io sunsetted its free Hobby plan in October 2024. New accounts get a one-time $5 trial credit, then pay-as-you-go. A single always-on 1 GB VM costs ~$6/month. Not free.

2.3 Why Oracle Cloud Always Free?

Feature	Oracle Always Free	Render Free	Fly.io
Always-on VM RAM	**24 GB (Ampere A1)**	512 MB (web service)	n/a (pay-as-you-go)
Postgres free expiry	n/a (we use Neon separately)	30 days ⚠️	n/a
Spin-down on idle	**No**	Yes (15 min) ⚠️	Optional
SMTP allowed	**Yes**	No ⚠️	Yes
Cost	**$0 forever**	$0 (with limits)	~$6/mo+
Catch	Signup approval friction; "out of capacity" in popular regions	Severe limits	No free tier

The Ampere A1 Flex shape offers **up to 4 OCPU + 24 GB RAM always-on for free, forever**. This is the only free tier in the industry with enough RAM to run ICE's heavy worker comfortably.

2.4 Why Neon for Postgres?

- **Never expires** on the free tier (Render's expires in 30 days).
- **pgvector extension supported** on free tier.
- **Serverless autoscaling + autosuspend** — perfect for a low-traffic POC.
- 0.5 GB storage is plenty for the baseline migration (~15 tables, mostly empty for a demo).
- Point-in-Time Recovery to 7 days included.

2.5 Why Upstash for Redis?

- TLS-native (`rediss://`) — required for cross-network Celery broker.
- **Up to 10 free Redis DBs** per account — enough for ICE's 3 logical DBs (cache, broker, result backend).
- 10K commands/day free. **See §9 for a Celery polling mitigation** if this is exceeded.
- Serverless, no VM to manage.

2.6 Why Cloudflare R2?

- **S3-compatible API** — drop-in for ICE's existing `boto3` client.
- **Zero egress fees** — critical because ICE streams videos via presigned URLs.
- 10 GB storage free + 1M Class A ops + 10M Class B ops/month.
- Drop-in: set `S3_ENDPOINT` to the R2 URL, `S3_USE_PATH_STYLE=false` .

2.7 Why Vercel for the frontend?

- Native Next.js host — detects `apps/web/next.config.js` , runs `next build` , serves standalone output on a global CDN.
- The `rewrites()` block in `next.config.js` (proxying `/api/*` to `API_URL`) is honored server-side.
- Free tier: 100 GB bandwidth, 100 GB-hours build, no spin-down.
- Auto-deploys on push to `main` via native GitHub integration — no Actions YAML needed for the frontend.

2.8 Why disable Judge0?

Judge0 requires **privileged Docker** (Docker-in-Docker) to sandbox untrusted code. This is: - Not supported on Render/Fly free tiers. - A security and operational burden on a shared VM. - Overkill for a private POC where you trust the code being executed.

ICE already implements a zero-regression fallback: when `SANDBOX_BACKEND=subprocess` (the default), the `/api/v1/execute` endpoint runs Python via a local `subprocess` with CPU/memory/time limits. This is fine for demos but **is a security risk for multi-user production** — only run code you trust.

---

3. Service Sign-Up & Provisioning

3.1 Oracle Cloud Always Free VM

Sign up 1. Go to https://signup.cloud.oracle.com/. 2. Use a credit card for identity verification — **it will never be charged** while you stay within Always Free limits. 3. Pick a **home region** with available A1 capacity. If US-East/EU-Frankfurt show "out of capacity", try **Mumbai**, **Johannesburg**, **Osaka**, or **Singapore**. You cannot change the home region later, so choose carefully.

Provision the VM 1. Console → **Compute** → **Instances** → **Create Instance**. 2. **Name:** `ice-prod` 3. **Image:** Canonical Ubuntu 22.04 (confirm the aarch64 variant). 4. **Shape:** `VM.Standard.A1.Flex` → set **4 OCPU** + **24 GB RAM** (the Always Free maximum). 5. **Networking:** default new VCN, public subnet, **assign a public IPv4 address**. 6. **SSH keys:** "Save private key" → store as `ice-vm.key` locally. **Do not lose this** — Oracle does not let you reset it. 7. Click **Create**. Wait ~2 min for the instance to be `RUNNING`.

Open firewall ports 1. Console → **Networking** → **Virtual Cloud Networks** → → **Security Lists** → **Default Security List**. 2. **Add Ingress Rules:** - `0.0.0.0/0` → `80/tcp` (Caddy HTTP-01 challenge) - `0.0.0.0/0` → `443/tcp` (HTTPS) - `<your-IP>/32` → `22/tcp` (SSH — restrict to your IP if possible; otherwise `0.0.0.0/0` with strong key only)

First login & bootstrap

```
# Locally (Windows PowerShell) — protect the key
icacls ice-vm.key /inheritance:r
icacls ice-vm.key /grant:r "$($env:USERNAME):(R)"
# (Linux/macOS): chmod 600 ice-vm.key

ssh -i ice-vm.key ubuntu@<VM_PUBLIC_IP>
```

On the VM:

```
# Install Docker Engine + compose plugin (Ubuntu ARM)
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=arm64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update
sudo apt-get install -y \
  docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin \
  postgresql-client

sudo usermod -aG docker $USER
newgrp docker

# Deploy directory
sudo mkdir -p /opt/ice && sudo chown ubuntu:ubuntu /opt/ice

# Firewall (defense in depth)
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
```

3.2 Neon (PostgreSQL 16 + pgvector)

Sign up 1. Go to https://console.neon.tech → sign up with GitHub. 2. Create a project named `ice-prod`. 3. Choose region **AWS US East (Ohio)** or matching the VM region for lowest latency.

Provision 1. Copy the **Pooled connection string** (ends with `...neon.tech/ice?sslmode=require`). The `-pooler` host is important — it handles many short-lived Celery connections. 2. Open the Neon **SQL Editor** and run: `sql CREATE EXTENSION IF NOT EXISTS vector; CREATE EXTENSION IF NOT EXISTS "uuid-ossp"; CREATE EXTENSION IF NOT EXISTS pg_trgm;` 3. Save the connection string — you'll use it as `DATABASE_URL`.

Run migrations from your local machine (one time):

```
$env:DATABASE_URL = "postgresql+asyncpg://USER:PASS@ep-xxx-pooler.us-east-2.aws.neon.tech/ice?sslmode=require"
uv run alembic -c db/alembic.ini upgrade head
```

3.3 Upstash (Redis)

1. Go to https://console.upstash.com → sign up.
2. Create **three** free Redis databases (Upstash allows up to 10 per account):
   - `ice-cache` — used for `REDIS_URL`
   - `ice-broker` — used for `CELERY_BROKER_URL`
   - `ice-result` — used for `CELERY_RESULT_BACKEND`
3. For each, choose **Global** (multi-region) or a single region matching Neon.
4. Copy each database's **Endpoint** and **Password**, then build the URL:

   ```
   rediss://default:<PASSWORD>@<ENDPOINT>:<PORT>
   ```

   Note the `rediss://` scheme (TLS) — Upstash enforces TLS by default.

3.4 Cloudflare R2

Enable R2 1. Go to https://dash.cloudflare.com → **R2 Object Storage** → **Enable**. 2. Cloudflare charges a one-time $1 to verify the card on file. **You will not be billed** while within the free tier.

Create the bucket 1. **R2** → **Create bucket** → name `ice-artifacts` → region **Auto** (or WWRA).

Create an API token 1. **R2** → **Manage R2 API Tokens** → **Create API token**. 2. Permissions: **Object Read & Write**. 3. Bucket: `ice-artifacts` (or "Apply to all buckets"). 4. Copy: - **Access Key ID** - **Secret Access Key** - **Endpoint URL** (looks like `https://<ACCOUNT_ID>.r2.cloudflarestorage.com`)

3.5 Vercel (frontend)

```
npm install -g vercel
cd "D:\Genesys_Systems\Interactive Cirriculum Engine"
vercel link
```

When prompted: - **Project name:** `ice-web` - **Root directory:** `apps/web` - **Framework preset:** Next.js - **Build command:** (leave default — Vercel detects `pnpm`) - **Output directory:** (leave blank — Next.js standalone) - **Install command:** `pnpm install --frozen-lockfile`

Set environment variables (Vercel dashboard → Project → Settings → Environment Variables):

Variable	Value	Environment
`API_URL`	`https://<VM_PUBLIC_IP>.sslip.io`	Production, Preview
`NEXT_PUBLIC_API_URL`	`https://<VM_PUBLIC_IP>.sslip.io`	Production, Preview

> `API_URL` is server-only — used by the Next.js `rewrites()` to proxy `/api/*` to the API. `NEXT_PUBLIC_API_URL` is exposed to the browser and used **only** for OAuth full-page redirects (see `apps/web/src/lib/auth.ts:119`).

3.6 UptimeRobot (keep-alive + monitoring)

1. Sign up at https://uptimerobot.com (free).
2. **Add New Monitor:**
   - Monitor type: **HTTP(s)**
   - Friendly name: `ICE API`
   - URL: `https://<VM_PUBLIC_IP>.sslip.io/api/health` (set after VM is up in §11 Phase C)
   - Interval: **5 minutes**
3. (Optional) Configure alert contacts (email, Discord webhook, Telegram).

3.7 (Optional) Supporting services

- **Groq API key** — free tier at https://console.groq.com. The primary LLM provider for ICE (model `llama-3.3-70b-versatile`).
- **Brevo SMTP** — free 300 emails/day at https://www.brevo.com. Use for email verification + support tickets. Get an SMTP key; host is `smtp-relay.brevo.com`, port 587.
- **Sentry** — free 5K errors/mo at https://sentry.io. Set `SENTRY_DSN` in `/opt/ice/.env`.
- **GitHub Container Registry PAT** — https://github.com/settings/tokens → Fine-grained token with `packages:write` scope on the repo.

---

4. Codebase Changes

All changes go on a new branch `feat/prod-deploy`.

4.1 Add `MINIO_EXTERNAL_ENDPOINT` to `.env.example`

Open `.env.example` and add after the `S3_*` block:

```
# ── Browser-facing S3 endpoint ──────────────────────────────
# Used to sign presigned URLs that browsers can resolve. On dev =
# http://localhost:9000; on prod = the public R2 endpoint URL.
# Required in 3 code paths: routers/curricula.py, tasks/signal_video.py, tasks/recap.py.
MINIO_EXTERNAL_ENDPOINT=http://localhost:9000
```

> This variable is read via `os.getenv("MINIO_EXTERNAL_ENDPOINT", "http://localhost:9000")` in three places but was never documented. Fixing this gap.

4.2 Create `infra/compose/docker-compose.prod.yml`

This file replaces the dev compose for production. Key differences from `docker-compose.dev.yml`: - No `postgres` / `redis` / `minio` / `judge0` services (external managed services now). - No bind-mounts of source code — uses built images from GHCR. - Adds a Caddy reverse proxy for TLS. - Uses `env_file` instead of inline env vars.

```yaml
# Production single-VM stack: api + worker + caddy on Oracle A1.
# External dependencies (Postgres, Redis) are managed services (Neon, Upstash).
# Run from /opt/ice on the VM.
name: ice-prod

services:
  api:
    image: ghcr.io/${GHCR_OWNER}/ice/api:${IMAGE_TAG:-latest}
    restart: unless-stopped
    env_file: /opt/ice/.env
    expose:
      - "8000"
    networks:
      - ice-net
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8000/health"]
      interval: 30s
      timeout: 5s
      retries: 5
      start_period: 30s

  worker:
    image: ghcr.io/${GHCR_OWNER}/ice/worker:${IMAGE_TAG:-latest}
    restart: unless-stopped
    env_file: /opt/ice/.env
    networks:
      - ice-net
    # Worker needs more memory for Whisper + Playwright + ffmpeg + Node 22 + Remotion.
    # 4 GB hard limit, 2 GB soft reservation — well within the 24 GB A1 quota.
    mem_limit: 4g
    mem_reservation: 2g
    # Reduce Redis polling to stay under Upstash 10K cmds/day free quota.
    # --without-gossip/-mingle/-heartbeat avoids several periodic Redis round-trips.
    # -s 30 = only one scheduler tick per 30 s (vs default 5 s).
    command: >
      uv run celery -A ice_worker.celery_app worker
      -l info
      --without-gossip --without-mingle --without-heartbeat
      -s 30

  caddy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - ice-net
    depends_on:
      - api

networks:
  ice-net:
    driver: bridge

volumes:
  caddy_data:
  caddy_config:
```

4.3 Create `infra/docker/Caddyfile`

Caddy automatically obtains and renews Let's Encrypt TLS certificates for the configured hostname. Using `<VM_IP>.sslip.io` gives a valid public hostname without owning a domain.

```
# Replace <VM_PUBLIC_IP> with your Oracle VM's public IPv4 address.
# sslip.io resolves <IP>.sslip.io to <IP>, so Caddy can complete the HTTP-01 challenge.

<VM_PUBLIC_IP>.sslip.com {
    encode zstd gzip

    # Health check — public, used by UptimeRobot keep-alive
    handle /api/health {
        reverse_proxy api:8000
    }

    # All /api/* requests (REST API)
    handle_path /api/* {
        reverse_proxy api:8000
    }

    # OpenAPI docs + spec (optional — comment out for production demos)
    handle /docs* { reverse_proxy api:8000 }
    handle /redoc* { reverse_proxy api:8000 }
    handle /openapi.json { reverse_proxy api:8000 }

    # OAuth callback paths (some flows hit /api/v1/auth/* directly)
    handle /api/v1/auth/* {
        reverse_proxy api:8000
    }

    # Fallback for anything else — frontend is on Vercel
    respond "ICE frontend is on Vercel. See DEPLOYMENT.md." 200
}
```

> **If you own a custom domain:** replace `<VM_PUBLIC_IP>.sslip.com` with your domain, add an A record pointing at the VM IP, and Caddy handles the rest identically.

4.4 Patch `infra/docker/Dockerfile.api`

The current API image doesn't ship `alembic` (it's in the dev dependency group) or `postgresql-client`. We need both so migrations can run as a pre-deploy step (in CI and on the VM).

Open `infra/docker/Dockerfile.api` and modify the apt-get install block to include `postgresql-client`:

```
RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential libpq-dev ffmpeg curl ca-certificates \
        postgresql-client \
    && rm -rf /var/lib/apt/lists/*
```

Then, after the `uv sync --no-dev` line, install `alembic` into the runtime venv:

```
RUN uv sync --no-dev && uv pip install alembic
```

> This keeps `alembic` in the runtime image without polluting the dev dependency group. The CI `migrate` job will use this same image to run `alembic upgrade head` against Neon.

4.5 Create `infra/compose/migrate.sh`

A small helper that runs Alembic inside the API container (uses the same env file so `DATABASE_URL` is correct):

```
#!/usr/bin/env bash
# Run Alembic migrations against the configured DATABASE_URL.
# Usage: IMAGE_TAG=<sha> GHCR_OWNER=<owner> bash migrate.sh
set -euo pipefail

: "${IMAGE_TAG:?must be set}"
: "${GHCR_OWNER:?must be set}"

docker run --rm \
  --env-file /opt/ice/.env \
  --network ice-prod_ice-net \
  "ghcr.io/${GHCR_OWNER}/ice/api:${IMAGE_TAG}" \
  sh -c "cd /app && uv run alembic -c db/alembic.ini upgrade head"
```

Make it executable: `chmod +x infra/compose/migrate.sh`.

4.6 Create `/opt/ice/.env` on the VM

SSH into the VM, then create `/opt/ice/.env` from the template below. Fill in **every** value.

```
# ═══ ICE Production Environment ═══
# Generate secrets with: openssl rand -hex 32

ENV=production
LOG_LEVEL=INFO
APP_NAME=ice
FRONTEND_URL=https://ice-web.vercel.app
CORS_ORIGINS=https://ice-web.vercel.app

# ── Database (Neon) ──────────────────────────────────
DATABASE_URL=postgresql+asyncpg://USER:PASS@ep-xxx-pooler.us-east-2.aws.neon.tech/ice?sslmode=require
DB_RLS_ENABLED=true

# ── Redis (Upstash) — 3 free DBs ─────────────────────
REDIS_URL=rediss://default:PASS@eu1-xxx-cache.upstash.io:6379
CELERY_BROKER_URL=rediss://default:PASS@eu1-xxx-broker.upstash.io:6379
CELERY_RESULT_BACKEND=rediss://default:PASS@eu1-xxx-result.upstash.io:6379

# ── Object storage (Cloudflare R2) ───────────────────
S3_ENDPOINT=https://<ACCOUNT_ID>.r2.cloudflarestorage.com
S3_ACCESS_KEY=<R2_ACCESS_KEY>
S3_SECRET_KEY=<R2_SECRET_KEY>
S3_REGION=auto
S3_BUCKET=ice-artifacts
S3_USE_PATH_STYLE=false
MINIO_EXTERNAL_ENDPOINT=https://<ACCOUNT_ID>.r2.cloudflarestorage.com

# ── Sandbox (DISABLED — Python subprocess only) ──────
SANDBOX_BACKEND=subprocess
JUDGE0_URL=http://localhost:2358
SANDBOX_CPU_LIMIT=2
SANDBOX_MEMORY_LIMIT=262144
SANDBOX_TIME_LIMIT=5
SANDBOX_NETWORK_DISABLED=true

# ── CPU-only ASR/OCR ─────────────────────────────────
ASR_MODEL=tiny
ASR_DEVICE=cpu
ASR_COMPUTE_TYPE=int8
OCR_ENGINE=rapidocr
OCR_GPU_ENABLED=false
VISION_MAX_WORKERS=1
VISION_MAX_FRAMES=80
VISION_ONNX_INTRA_OP_THREADS=1

# ── Auth (ROTATE THESE — generate with `openssl rand -hex 32`) ─
JWT_SECRET=REPLACE_WITH_64_HEX_CHARS
JWT_ALGORITHM=HS256
JWT_ACCESS_TTL_MIN=60
JWT_REFRESH_TTL_DAYS=7
SSE_TOKEN_SECRET=REPLACE_WITH_DIFFERENT_64_HEX_CHARS

# ── OAuth (fill if you use Google/GitHub login) ──────
GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=
GOOGLE_OAUTH_REDIRECT_URI=https://<VM_PUBLIC_IP>.sslip.io/api/v1/auth/google/callback
GITHUB_OAUTH_CLIENT_ID=
GITHUB_OAUTH_CLIENT_SECRET=
GITHUB_OAUTH_REDIRECT_URI=https://<VM_PUBLIC_IP>.sslip.io/api/v1/auth/github/callback

# ── Email (Brevo SMTP — 300/day free) ────────────────
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=<BREVO_LOGIN_EMAIL>
SMTP_PASSWORD=<BREVO_SMTP_KEY>
FROM_EMAIL=noreply@<your-domain>
SUPPORT_EMAIL=you@<your-domain>

# ── LLM ──────────────────────────────────────────────
GROQ_API_KEY=<your free Groq key>
GROQ_MODEL=llama-3.3-70b-versatile
GROQ_MAX_RETRIES=5
GROQ_BACKOFF_INITIAL=2.0
OPENAI_API_KEY=
OPENAI_MODEL_PRIMARY=gpt-4o
OPENROUTER_API_KEY=

# ── Observability ────────────────────────────────────
SENTRY_DSN=
PROMETHEUS_METRICS_PORT=9090

# ── Pipeline ─────────────────────────────────────────
PIPELINE_RUN_TESTS=false
PIPELINE_PREFER_CAPTIONS=true
PIPELINE_UPLOAD_MAX_BYTES=536870912
PIPELINE_UPLOAD_ALLOWED_EXTS=.mp4,.mov,.mkv,.webm,.avi,.m4v
```

```
PIPELINE_MAX_VIDEO_DURATION_SEC=3600
PIPELINE_MIN_VIDEO_DURATION_SEC=30
PIPELINE_CHUNK_WINDOW_SEC=300

# ── Compose-level (used by docker-compose.prod.yml) ──────────
IMAGE_TAG=latest
GHCR_OWNER=<your-github-username-or-org>
```

4.7 Patch `.github/workflows/deploy.yml`

Replace the `deploy-staging` and `deploy-production` jobs' `echo TODO` steps with real migrate + deploy logic. The full replacement for the `deploy-staging` job (which is what we use for the POC) is:

```yaml
  migrate:
    name: Apply migrations
    needs: build-and-push
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Install uv
        uses: astral-sh/setup-uv@v3
        with:
          version: "latest"
      - name: Sync dependencies (api only)
        run: uv sync --no-dev --package ice-api
      - name: Run alembic upgrade head against Neon
        env:
          DATABASE_URL: ${{ secrets.NEON_DATABASE_URL }}
        run: uv run alembic -c db/alembic.ini upgrade head

  deploy-vm:
    name: Deploy to Oracle VM
    needs: migrate
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Copy compose + Caddyfile to VM
        uses: appleboy/scp-action@v0.1.7
        with:
          host: ${{ secrets.VM_HOST }}
          username: ubuntu
          key: ${{ secrets.VM_SSH_KEY }}
          source: "infra/compose/docker-compose.prod.yml,infra/docker/Caddyfile,infra/compose/migrate.sh"
          target: "/opt/ice/"
          strip_components: 2
      - name: Pull images and restart on VM
        uses: appleboy/ssh-action@v1.0.3
        env:
          IMAGE_TAG: ${{ github.sha }}
          GHCR_OWNER: ${{ github.repository_owner }}
        with:
          host: ${{ secrets.VM_HOST }}
          username: ubuntu
          key: ${{ secrets.VM_SSH_KEY }}
          envs: IMAGE_TAG,GHCR_OWNER
          script: |
            cd /opt/ice
            chmod +x migrate.sh
            echo "${{ secrets.GHCR_PAT }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin
            docker compose -f docker-compose.prod.yml pull
            docker compose -f docker-compose.prod.yml up -d --remove-orphans
            # Run migrations inside the freshly pulled api image
            IMAGE_TAG=$IMAGE_TAG GHCR_OWNER=$GHCR_OWNER bash migrate.sh
            docker image prune -f
            echo "Deployed $IMAGE_TAG"
```

> **Note on `strip_components: 2`**: the `scp-action` flattens `infra/compose/docker-compose.prod.yml` → `docker-compose.prod.yml` in `/opt/ice/`. Adjust if your paths differ. The Caddyfile is copied to `/opt/ice/Caddyfile`, which `docker-compose.prod.yml` mounts at `./Caddyfile`.

4.8 GitHub Actions secrets to set

Go to the GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**:

Secret name	Value	Used by
`NEON_DATABASE_URL`	`postgresql+asyncpg://…@…neon.tech/ice?sslmode=require`	`migrate` job
`VM_HOST`	`<VM_PUBLIC_IP>`	`deploy-vm` job
`VM_SSH_KEY`	contents of `ice-vm.key` (the private key)	`deploy-vm` job
`GHCR_PAT`	fine-grained PAT with `packages:write`	`deploy-vm` job

4.9 Vercel environment variables

Set in the Vercel project (see §3.5). These are **separate** from GitHub Actions secrets because Vercel is a different runtime environment.

---

5. Environment Variable Mapping

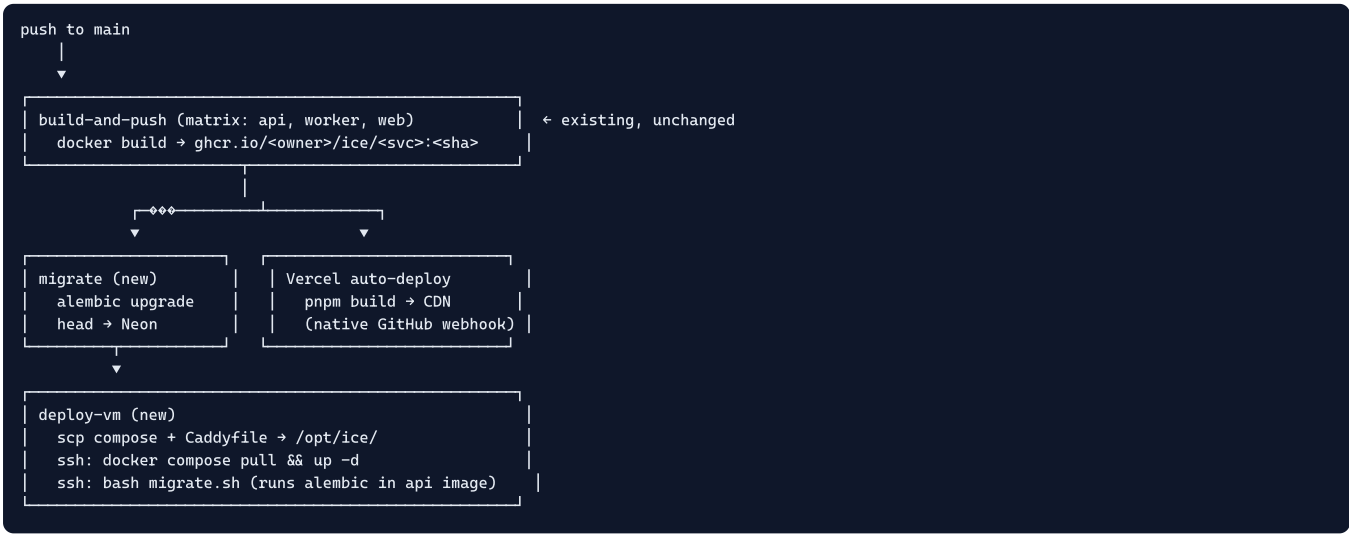
Complete mapping from local dev values to production values.

Variable	Local dev (compose)	Production	S
`ENV`	`dev`	`production`	h
`DATABASE_URL`	`postgresql+asyncpg://ice:ice_dev_password@postgres:5432/ice`	`postgresql+asyncpg://…@…neon.tech/ice?sslmode=require`	N U
`DB_RLS_ENABLED`	`true`	`true`	u
`REDIS_URL`	`redis://redis:6379/0`	`rediss://default:PASS@…-cache.upstash.io:6379`	U #
`CELERY_BROKER_URL`	`redis://redis:6379/1`	`rediss://default:PASS@…-broker.upstash.io:6379`	U #
`CELERY_RESULT_BACKEND`	`redis://redis:6379/2`	`rediss://default:PASS@…-result.upstash.io:6379`	U #
`S3_ENDPOINT`	`http://minio:9000`	`https://<ACCT>.r2.cloudflarestorage.com`	R
`S3_ACCESS_KEY`	`ice_minio`	R2 Access Key ID	R
`S3_SECRET_KEY`	`ice_minio_secret`	R2 Secret Access Key	R
`S3_REGION`	`us-east-1`	`auto`	R
`S3_BUCKET`	`ice-artifacts`	`ice-artifacts`	u
`S3_USE_PATH_STYLE`	`true`	**`false`**	R v h
`MINIO_EXTERNAL_ENDPOINT`	`http://localhost:9000`	`https://<ACCT>.r2.cloudflarestorage.com`	R (l fa
`JUDGE0_URL`	`http://judge0:2358`	(unused)	Ju d
`SANDBOX_BACKEND`	`subprocess` (default)	`subprocess`	e Ju d
`ASR_MODEL`	`large-v3` (default)	**`tiny`**	C fi R
`ASR_DEVICE`	`cuda` (default)	**`cpu`**	C
`ASR_COMPUTE_TYPE`	`int8_float16` (default)	**`int8`**	C
`OCR_ENGINE`	`rapidocr` (default)	`rapidocr`	u ((A c
`OCR_GPU_ENABLED`	`false` (default)	`false`	C
`VISION_MAX_WORKERS`	`0` (default = auto)	**`1`**	b u
`VISION_MAX_FRAMES`	`150` (default)	**`80`**	re C p
`CORS_ORIGINS`	`http://localhost:3000`	`https://ice-web.vercel.app`	V
`FRONTEND_URL`	`http://localhost:3000`	`https://ice-web.vercel.app`	C li
`JWT_SECRET`	`change_me_in_prod`	`openssl rand -hex 32` output	**r**
`SSE_TOKEN_SECRET`	`""` (falls back to JWT)	`openssl rand -hex 32` output	**r**
`GOOGLE_OAUTH_REDIRECT_URI`	`http://localhost:8000/api/v1/auth/google/callback`	`https://<VM_IP>.sslip.io/api/v1/auth/google/callback`	V
`GITHUB_OAUTH_REDIRECT_URI`	`http://localhost:8000/api/v1/auth/github/callback`	`https://<VM_IP>.sslip.io/api/v1/auth/github/callback`	V
`SMTP_HOST`	`smtp.gmail.com` (default)	`smtp-relay.brevo.com`	B
`API_URL` (Vercel)	n/a	`https://<VM_IP>.sslip.io`	N re

Variable	Local dev (compose)	Production	S
`NEXT_PUBLIC_API_URL` (Vercel)	n/a	`https://<VM_IP>.sslip.io`	C re

6. CI/CD Pipeline

6.1 Flow



6.2 Triggers

- **Frontend (Vercel):** every push to `main` (native GitHub integration — no Actions config needed).
- **Backend + worker (GitHub Actions):** every push to `main` runs `build-and-push` → `migrate` → `deploy-vm`.
- **Manual:** `workflow_dispatch` from the Actions tab lets you redeploy without a commit.
- **Releases:** optionally gate production behind `release/v*` tags (already scaffolded in `deploy.yml`).

6.3 Secrets management

- **GitHub Actions secrets** — `NEON_DATABASE_URL`, `VM_HOST`, `VM_SSH_KEY`, `GHCR_PAT`. Set in repo Settings → Secrets and variables → Actions.
- **Vercel env vars** — `API_URL`, `NEXT_PUBLIC_API_URL`. Set in Vercel dashboard.
- **VM env file** — `/opt/ice/.env` (chmod 600). Contains all other secrets (LLM keys, SMTP, R2, JWT). Never committed to git.
- **Runtime secrets** — JWT, SSE, OAuth secrets are in `/opt/ice/.env`, mounted into both `api` and `worker` containers.

7. Persistent Data & Backups

7.1 Data residency

Data	Where	Durability
Postgres (curricula, users, exercises)	Neon	11 nines, PITR 7 days free
Uploaded videos, transcripts, audio	Cloudflare R2	11 nines, zero egress
Celery task state	Upstash	replicated; ephemeral anyway
Caddy TLS certs	VM volume `caddy_data`	auto-regenerable if lost
API/worker ephemeral logs	VM (docker json-file)	lost on container recreate

7.2 Daily DB backup to R2 (belt-and-suspenders)

Even though Neon has PITR, take a daily logical backup for extra safety. Create `/opt/ice/backup.sh` on the VM:

```
#!/usr/bin/env bash
# Daily Postgres backup → R2. Retains 30 days.
# Add to crontab: 0 3 * * * /opt/ice/backup.sh >> /var/log/ice-backup.log 2>&1
set -euo pipefail

source /opt/ice/.env

TS=$(date -u +%Y%m%dT%H%M%SZ)
KEY="backups/${TS}.sql.gz"

# Stream pg_dump | gzip | s3 put — no local disk used
pg_dump "$DATABASE_URL" | gzip | \
  python3 -c "
import os, sys, boto3
c = boto3.client('s3',
    endpoint_url=os.environ['S3_ENDPOINT'],
    aws_access_key_id=os.environ['S3_ACCESS_KEY'],
    aws_secret_access_key=os.environ['S3_SECRET_KEY'],
    region_name=os.environ.get('S3_REGION', 'auto'))
c.put_object(Bucket=os.environ['S3_BUCKET'], Key='${KEY}', Body=sys.stdin.buffer)
print('Uploaded ${KEY}')
"

# Prune backups older than 30 days
python3 -c "
import os, boto3
from datetime import datetime, timedelta
c = boto3.client('s3',
    endpoint_url=os.environ['S3_ENDPOINT'],
    aws_access_key_id=os.environ['S3_ACCESS_KEY'],
    aws_secret_access_key=os.environ['S3_SECRET_KEY'],
    region_name=os.environ.get('S3_REGION', 'auto'))
resp = c.list_objects_v2(Bucket=os.environ['S3_BUCKET'], Prefix='backups/')
cutoff = datetime.utcnow() - timedelta(days=30)
for o in resp.get('Contents', []):
    try:
        d = datetime.strptime(o['Key'].split('/')[-1].split('T')[0], '%Y%m%d')
    except ValueError:
        continue
    if d < cutoff:
        c.delete_object(Bucket=os.environ['S3_BUCKET'], Key=o['Key'])
        print(f'Deleted {o[\"Key\"]}')
"
```

Make executable and install:

```
chmod +x /opt/ice/backup.sh
sudo apt-get install -y python3-boto3
( sudo crontab -l 2>/dev/null; echo "0 3 * * * /opt/ice/backup.sh >> /var/log/ice-backup.log 2>&1" ) | sudo crontab -
```

7.3 Restore procedure

```
# List backups
source /opt/ice/.env
python3 -c "import os,boto3;
        c=boto3.client('s3',endpoint_url=os.environ['S3_ENDPOINT'],aws_access_key_id=os.environ['S3_ACCESS_KEY'],aws_secret_access_key=os.enviro
        [print(o['Key']) for o in c.list_objects_v2(Bucket=os.environ['S3_BUCKET'],Prefix='backups/').get('Contents',[])]"

# Restore: download → gunzip → psql
python3 -c "import os,boto3;
        c=boto3.client('s3',endpoint_url=os.environ['S3_ENDPOINT'],aws_access_key_id=os.environ['S3_ACCESS_KEY'],aws_secret_access_key=os.enviro
        c.download_file(os.environ['S3_BUCKET'],'backups/20260101T030000Z.sql.gz','/tmp/restore.sql.gz')"
gunzip -c /tmp/restore.sql.gz | psql "$DATABASE_URL"
```

---

8. Monitoring & Logging

Signal	Tool	Cost	Setup
Uptime + keep-alive	**UptimeRobot**	$0	Monitor `GET /api/health` every 5 min (§3.6)
App errors	**Sentry**	$0 (5K err/mo)	Set `SENTRY_DSN` in `/opt/ice/.env`; ICE already has Sentry integration
Container logs	`docker compose logs -f`	$0	SSH to VM; rotate with `logrotate` (below)
DB query analytics	Neon dashboard	$0	Built-in
Redis metrics	Upstash dashboard	$0	Built-in
R2 usage	Cloudflare dashboard	$0	Built-in
VM CPU/mem	Oracle Cloud metrics	$0	Built-in
Health endpoint	`GET /api/health`	$0	Already implemented in `ice_api.main`

8.1 Log rotation on the VM

Docker json-file logs grow unbounded by default. Configure rotation:

```
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
EOF
sudo systemctl restart docker
```

8.2 Health check

ICE's API exposes `GET /api/health` (defined in `apps/api/src/ice_api/main.py`). UptimeRobot pings this every 5 min, which: 1. Confirms the API is alive. 2. Prevents any platform-level idle spin-down (not relevant on Oracle always-on, but harmless).

8.3 (Optional) Centralized logging with Grafana Cloud Free

If you want searchable logs beyond `docker compose logs`, sign up at https://grafana.com (free: 50 GB logs, 3 dashboards). Add a Grafana Alloy agent on the VM to ship `dockerd` logs to Loki. Out of scope for the basic POC.

---

9. Limitations & Mitigations

Limitation	Impact	Mitigation
`SANDBOX_BACKEND=subprocess` runs untrusted Python on the API host	**Security risk** — code executes with the API process's privileges	Only run code you trust in the POC. For multi-user production, deploy Judge0 on a separate small VM later. The code already has zero-regression fallback (`ice_shared.judge0_client.run_sandbox`).
Oracle signup may fail / show "out of capacity"	Blocks deploy entirely	Fallback to Azure for Students ($100 credit, no signup friction) — see §12. Or DigitalOcean via GitHub Student Pack ($200 credit ≈ 16 months).
ARM64 architecture	Possible dep incompatibility	All ICE deps ship aarch64 builds: Playwright (official ARM Linux), Whisper (CPU), ONNX Runtime (official ARM), RapidOCR (ONNX-based), Node 22 (official ARM), Remotion (Node). Verify on first deploy with `docker compose logs worker` .
Neon auto-suspends after ~5 min idle	~300 ms cold-start on first query	Acceptable for POC. Upgrade to always-on ($19/mo) if demos are latency-sensitive.
Upstash 10K cmds/day	Celery polling `redis.brpop` can exceed this on a busy worker	The `docker-compose.prod.yml` worker command uses `--without-gossip --without-mingle --without-heartbeat -s 30` to reduce Redis round-trips to ~3K/day. If you still exceed 10K, upgrade Upstash to Pay-As-You-Go (~$0.50/mo realistically) or self-host Redis on the VM (free, more ops).
Free R2 is 10 GB storage	Limits stored video count	`PIPELINE_UPLOAD_MAX_BYTES=512MB` ≈ 20 short videos. Archive or delete old artifacts manually. Upgrade R2 ($0.015/GB-month) if needed.
Free Vercel bandwidth is 100 GB	Demos with many video streams could approach limit	Videos stream directly from R2 (presigned URLs), **not** through Vercel — so Vercel bandwidth only covers HTML/JS, not video. Effectively unlimited for POC.
Email from the VM	Need working SMTP for email verification	VM has no port restrictions — use Brevo (300/day free) or Gmail App Passwords. Render free would block this (§2.1).
Single VM = SPOF	Brief downtime on restart/redeploy	Acceptable for POC. `docker compose up -d` is ~10 s downtime. Backups (§7) cover data loss.
API self-heals drifted columns on startup	Schema can drift from Alembic baseline	Run `alembic upgrade head` on every deploy (handled by `migrate.sh`). The self-heal in `main.py` is best-effort and not a substitute.
`alembic autogenerate` not wired	New migrations must be hand-written	Already the case in dev. Use `make migrate-new m="..."` then edit the generated file.

10. Cost Estimate

10.1 Monthly cost breakdown (POC usage)

Service	Free quota	ICE POC usage	Overage cost
Oracle A1 VM	4 OCPU + 24 GB always-on	4 OCPU + ~6 GB	**$0** (hard cap, never charged within free tier)
Neon Postgres	0.5 GB + 100 compute hrs/mo	~50 MB	$0
Upstash Redis (free)	10K cmds/day, 256 MB each	~3-30K cmds/day	$0 or ~$0.50/mo (Pay-As-You-Go if exceeded)
Vercel	100 GB bandwidth, 100 GB-hrs build	<5 GB (video streams via R2, not Vercel)	$0
Cloudflare R2	10 GB storage, 0 egress, 1M Class A + 10M Class B ops/mo	2-8 GB	$0
GHCR	1 GB private / unlimited public	Public repo	$0
UptimeRobot	50 monitors @ 5-min	1	$0
Sentry	5K errors/mo	<1K	$0
Brevo SMTP	300 emails/day	<10	$0
Realistic total			**$0 - $0.50/month**

10.2 Ceiling estimate (if POC grows)

Scenario	Monthly cost
10 GB DB on Neon (exceeds free 0.5 GB)	$19/mo
50 GB video on R2 (exceeds free 10 GB)	$0.60/mo
50 GB egress via Vercel (exceeds free 100 GB)	$0 (video goes via R2)
Upstash Redis 100K cmds/day	~$1/mo
Ceiling total	**~$20-25/month**

10.3 One-time costs

Item	Cost
Oracle Cloud identity verification (card auth hold)	$0 (released)
Cloudflare R2 enablement (card auth)	$1 (refundable)
Domain name (optional)	$10-15/year

---

11. Execution Checklist

Phase A — Signups & provisioning (run in parallel)

- [ ] **A1** Create Oracle Cloud account at https://signup.cloud.oracle.com/
- [ ] **A2** Pick a home region with A1 capacity (try Mumbai/Johannesburg/Osaka if US/EU full)
- [ ] **A3** Provision A1 Flex VM: Ubuntu 22.04 ARM, **4 OCPU + 24 GB RAM**, public IP
- [ ] **A4** Save SSH private key as `ice-vm.key` (protect it: `chmod 600` / `icacls`)
- [ ] **A5** Open ports 22/80/443 in the VCN Security List
- [ ] **A6** SSH into the VM, run the Docker bootstrap snippet (§3.1)
- [ ] **A7** Sign up at **Neon** (https://console.neon.tech), create project `ice-prod`
- [ ] **A8** Copy the **pooled** connection string
- [ ] **A9** In Neon SQL Editor: `CREATE EXTENSION vector; CREATE EXTENSION "uuid-ossp"; CREATE EXTENSION pg_trgm;`
- [ ] **A10** Sign up at **Upstash** (https://console.upstash.com), create 3 free Redis DBs (cache/broker/result)
- [ ] **A11** Copy each DB's `rediss://` URL
- [ ] **A12** Sign up at **Cloudflare** (https://dash.cloudflare.com), enable R2 ($1 card auth)

- [ ] **A13** Create R2 bucket `ice-artifacts`
- [ ] **A14** Create R2 API token (Object Read & Write), copy Access Key + Secret + Endpoint
- [ ] **A15** Sign up at **Vercel** (https://vercel.com), import the GitHub repo
- [ ] **A16** Configure Vercel project: root `apps/web`, framework Next.js
- [ ] **A17** Sign up at **UptimeRobot** (https://uptimerobot.com) — add monitor after Phase C
- [ ] **A18** (Optional) Get free **Groq** API key (https://console.groq.com)
- [ ] **A19** (Optional) Sign up at **Brevo** (https://www.brevo.com), get SMTP key
- [ ] **A20** (Optional) Sign up at **Sentry** (https://sentry.io), get DSN
- [ ] **A21** (Optional) Create GitHub fine-grained PAT with `packages:write` scope

Phase B — Codebase changes (commit to branch `feat/prod-deploy`)

- [ ] **B1** Add `MINIO_EXTERNAL_ENDPOINT` line to `.env.example`
- [ ] **B2** Create `infra/compose/docker-compose.prod.yml` (content in §4.2)
- [ ] **B3** Create `infra/docker/Caddyfile` with `<VM_PUBLIC_IP>.sslip.com` (§4.3)
- [ ] **B4** Patch `infra/docker/Dockerfile.api`: add `postgresql-client` + `uv pip install alembic` (§4.4)
- [ ] **B5** Create `infra/compose/migrate.sh` and `chmod +x` it (§4.5)
- [ ] **B6** Patch `.github/workflows/deploy.yml` with `migrate` + `deploy-vm` jobs (§4.7)
- [ ] **B7** Push branch, open PR, wait for CI (lint/typecheck/test) to pass
- [ ] **B8** Merge to `main` (this triggers the first image build → GHCR)

Phase C — VM bootstrap

- [ ] **C1** SSH into the VM: `ssh -i ice-vm.key ubuntu@<VM_PUBLIC_IP>`
- [ ] **C2** Create `/opt/ice/.env` from the template in §4.6, fill in **every** value
- [ ] **C3** Generate secrets: `openssl rand -hex 32` for `JWT_SECRET` and `SSE_TOKEN_SECRET`
- [ ] **C4** Set `IMAGE_TAG=<sha from latest GHCR build>` and `GHCR_OWNER=<your-github-username>` in `/opt/ice/.env`
- [ ] **C5** `chmod 600 /opt/ice/.env`
- [ ] **C6** Log Docker into GHCR: `echo "<PAT>" | docker login ghcr.io -u <user> --password-stdin`
- [ ] **C7** Copy compose + Caddyfile to VM (or wait for first deploy job to do it):

```
# On your local machine:
scp -i ice-vm.key infra/compose/docker-compose.prod.yml ubuntu@<VM_IP>:/opt/ice/
scp -i ice-vm.key infra/docker/Caddyfile ubuntu@<VM_IP>:/opt/ice/Caddyfile
scp -i ice-vm.key infra/compose/migrate.sh ubuntu@<VM_IP>:/opt/ice/migrate.sh
```

- [ ] **C8** First-time launch on the VM:

```
cd /opt/ice
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

- [ ] **C9** Run migrations:

```
IMAGE_TAG=<sha> GHCR_OWNER=<owner> bash migrate.sh
```

- [ ] **C10** Verify health:

```
curl -k https://<VM_PUBLIC_IP>.sslip.io/api/health
# Expected: 200 OK with JSON {"status":"healthy", ...}
```

> The first request triggers Caddy's Let's Encrypt cert issuance (may take 10-30 s).

- [ ] **C11** Check logs for errors:

```
docker compose -f docker-compose.prod.yml logs -f api worker caddy
```

Phase D — Frontend (Vercel)

- [ ] **D1** In Vercel dashboard, set env vars `API_URL` and `NEXT_PUBLIC_API_URL` to `https://<VM_PUBLIC_IP>.sslip.io`
- [ ] **D2** Trigger a Vercel redeploy (push any commit or click "Redeploy")

- [ ] **D3** Open the Vercel deployment URL — confirm the homepage loads
- [ ] **D4** (If OAuth enabled) Set callback URLs in Google/GitHub OAuth console:
  - `https://<VM_PUBLIC_IP>.sslip.io/api/v1/auth/google/callback`
  - `https://<VM_PUBLIC_IP>.sslip.io/api/v1/auth/github/callback`
- [ ] **D5** Test login flow end-to-end

Phase E — Networking & DNS (optional)

- [ ] **E1** Confirm `https://<VM_PUBLIC_IP>.sslip.io` works from a browser (lock icon = cert valid)
- [ ] **E2** (Optional) Buy a domain, add an A record pointing at the VM IP
- [ ] **E3** (Optional) Update `Caddyfile` with the real domain, `docker compose restart caddy`

Phase F — Smoke test

- [ ] **F1** Open the Vercel URL, log in
- [ ] **F2** Upload a short YouTube video URL via the web UI (or upload a small MP4 file)
- [ ] **F3** Watch the worker: `docker compose -f docker-compose.prod.yml logs -f worker`
  - Confirm: ingest → caption harvest (or Whisper tiny CPU) → OCR → segment → concepts → exercises → ready
- [ ] **F4** Open the generated curriculum in the UI — confirm segments, concepts, exercises render
- [ ] **F5** Click a video artifact — confirm the R2 presigned URL plays in-browser
- [ ] **F6** Try `/api/v1/execute` for a Python exercise — confirm subprocess sandbox returns stdout

Phase G — Ops hardening

- [ ] **G1** Add UptimeRobot monitor: `GET https://<VM_PUBLIC_IP>.sslip.io/api/health`, 5-min interval
- [ ] **G2** Install backup script:

```
sudo apt-get install -y python3-boto3
# Create /opt/ice/backup.sh from §7.2, chmod +x
( sudo crontab -l 2>/dev/null; echo "0 3 * * * /opt/ice/backup.sh >> /var/log/ice-backup.log 2>&1" ) | sudo crontab -
```

- [ ] **G3** Configure Docker log rotation (§8.1)
- [ ] **G4** Enable VM firewall: `sudo ufw allow 22,80,443/tcp && sudo ufw --force enable`
- [ ] **G5** Set GitHub Actions secrets: `NEON_DATABASE_URL`, `VM_HOST`, `VM_SSH_KEY`, `GHCR_PAT`
- [ ] **G6** Push a trivial commit to `main` → confirm the full deploy pipeline runs green
- [ ] **G7** Tag `release/v0.1.0-prod` to formally cut the production build

Phase H — Rollback plan

- [ ] **H1** To roll back the app: edit `/opt/ice/.env`, set `IMAGE_TAG=<previous-sha>`, then:

```
docker compose -f docker-compose.prod.yml pull && docker compose -f docker-compose.prod.yml up -d
```

- [ ] **H2** To roll back the DB:

```
uv run alembic -c db/alembic.ini downgrade -1
```

Reversibility is verified in CI (`db-migration-check` job in `ci.yml`).

- [ ] **H3** To restore DB from backup: see §7.3.

---

12. Recommended Fallback Architecture

If Oracle Cloud Always Free signup fails (common — "out of capacity" errors in popular regions, manual review delays), switch **only the VM platform**. Everything else (Neon, Upstash, R2, Vercel, GitHub Actions) stays identical.

12.1 Fallback Option A — Azure for Students

Element	Oracle (primary)	Azure for Students (fallback)
VM shape	Ampere A1 Flex, 4 OCPU + 24 GB	B1s, 1 vCPU + 1 GB RAM
Architecture	ARM64 (aarch64)	x86_64
Cost	$0 forever	$0 for 12 months, $100 credit covers overages
Signup	Card verified, never charged	`.edu` email, no card required
RAM risk	None (24 GB headroom)	**Tight** — may OOM under heavy pipeline
Required env tweaks	none	`ASR_MODEL=tiny`, `VISION_MAX_FRAMES=60`, `VISION_MAX_WORKERS=1`, `PIPELINE_MAX_VIDEO_DURATION_SEC=600`

Sign up: https://azure.microsoft.com/en-us/free/students/ with your school email. No credit card required. $100 credit + 12 months of free B1s VM (750 hrs/mo = always-on).

Provision: 1. Azure Portal → **Virtual machines** → **Create**. 2. Image: Ubuntu Server 22.04 LTS (x64). 3. Size: **Standard_B1s** (1 vCPU, 1 GB RAM). 4. Authentication: SSH public key. 5. Networking: allow inbound ports 22, 80, 443. 6. Create. SSH in and run the same Docker bootstrap snippet as §3.1 (use `amd64` instead of `arm64` in the apt source line).

Required env changes (in `/opt/ice/.env`):

```
ASR_MODEL=tiny
ASR_DEVICE=cpu
VISION_MAX_FRAMES=60
VISION_MAX_WORKERS=1
PIPELINE_MAX_VIDEO_DURATION_SEC=600   # 10 min max — CPU ASR on 1 GB is slow
```

> The worker's `mem_limit` in `docker-compose.prod.yml` should also be reduced to `1500m` to fit the 1 GB host (with swap). Add a swap file:
>
> ```
> sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
> echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
> ```

12.2 Fallback Option B — DigitalOcean via GitHub Student Pack

The GitHub Student Developer Pack (https://education.github.com/pack) includes a **$200 DigitalOcean credit**. This is enough for ~16 months of a 2 GB / 1 vCPU droplet (~$12/mo).

Element	DigitalOcean
VM shape	Basic Droplet, 1 vCPU + 2 GB RAM (x86_64)
Cost	$0 for ~16 months via $200 credit, then ~$12/mo
Architecture	x86_64 (no ARM compat worries)
RAM risk	Low (2 GB is comfortable for tiny ASR + reduced frames)
Required env tweaks	`ASR_MODEL=tiny`, `VISION_MAX_FRAMES=80`

Sign up: https://education.github.com/pack → verify student status → claim DigitalOcean credit.

Provision: 1. https://cloud.digitalocean.com → **Droplets** → **Create**. 2. Region: closest to Neon/Upstash. 3. Image: Ubuntu 22.04 (x64). 4. Size: **Basic / Shared CPU / $12/mo (1 vCPU, 2 GB, 50 GB SSD)**. 5. Authentication: SSH key. 6. Add the droplet IP to the GitHub `VM_HOST` secret. 7. SSH in and run the Docker bootstrap (use `amd64` apt source).

12.3 Fallback Option C — Render paid ($7-15/mo)

If you prefer the simplicity of PaaS over a VM, bump the worker to a paid Render instance:

Service	Render plan	Cost
API (web service)	Starter (512 MB)	$7/mo
Worker (background worker)	**Standard (2 GB)**	$15/mo
Postgres	Neon free (not Render — avoids 30-day expiry)	$0
Redis	Upstash free	$0
Frontend	Vercel free	$0
Total		**~$22/month**

This uses Render's `render.yaml` blueprint and requires no VM management, but costs money and the API still spins down (15 min idle) unless you also bump it to paid. For the worker, **Standard (2 GB) is required** — the Free/Starter (512 MB) will OOM.

12.4 Decision tree

```
Oracle signup succeeds?
├── Yes → Use Oracle Always Free ($0, 24 GB RAM). Done.
└── No  → Have a .edu email?
          ├── Yes → Azure for Students ($0, 1 GB RAM, tight). Done.
          └── No  → Have GitHub Student Pack?
                    ├── Yes → DigitalOcean $200 credit (~16 mo free, 2 GB). Done.
                    └── No  → Accept $22/mo on Render paid, OR co-locate on a $6 Hetzner CX22.
```

Appendix A — Quick reference: all files touched

File	Action	Section
`.env.example`	edit (add `MINIO_EXTERNAL_ENDPOINT`)	§4.1
`infra/compose/docker-compose.prod.yml`	**create**	§4.2
`infra/docker/Caddyfile`	**create**	§4.3
`infra/docker/Dockerfile.api`	edit (add `postgresql-client` + alembic)	§4.4
`infra/compose/migrate.sh`	**create**	§4.5
`.github/workflows/deploy.yml`	edit (add `migrate` + `deploy-vm` jobs)	§4.7

Appendix B — Quick reference: all secrets needed

Secret	Where stored	How generated
`JWT_SECRET`	`/opt/ice/.env`	`openssl rand -hex 32`
`SSE_TOKEN_SECRET`	`/opt/ice/.env`	`openssl rand -hex 32`
Neon DB password	Neon dashboard + `/opt/ice/.env`	Neon-generated
Upstash Redis passwords	Upstash dashboard + `/opt/ice/.env`	Upstash-generated
R2 Access Key + Secret	Cloudflare dashboard + `/opt/ice/.env`	R2 API token
Groq API key	`/opt/ice/.env`	https://console.groq.com
Brevo SMTP key	`/opt/ice/.env`	Brevo dashboard
OAuth client secrets	Google/GitHub consoles + `/opt/ice/.env`	OAuth provider
Oracle VM SSH key	local `ice-vm.key`	Oracle at VM creation
GHCR PAT	GitHub secrets + VM docker login	GitHub fine-grained token
`NEON_DATABASE_URL`	GitHub Actions secret	Neon pooled URL
`VM_HOST`	GitHub Actions secret	Oracle VM public IP
`VM_SSH_KEY`	GitHub Actions secret	contents of `ice-vm.key`

Appendix C — Common operations cheatsheet

```
# SSH into the VM
ssh -i ice-vm.key ubuntu@<VM_PUBLIC_IP>

# Tail all service logs
cd /opt/ice && docker compose -f docker-compose.prod.yml logs -f

# Tail one service
docker compose -f docker-compose.prod.yml logs -f api
docker compose -f docker-compose.prod.yml logs -f worker

# Restart a service
docker compose -f docker-compose.prod.yml restart api

# Roll back to a previous image
sed -i 's/^IMAGE_TAG=.*/IMAGE_TAG=<previous-sha>/' /opt/ice/.env
docker compose -f docker-compose.prod.yml pull && docker compose -f docker-compose.prod.yml up -d

# Run migrations manually
IMAGE_TAG=<sha> GHCR_OWNER=<owner> bash /opt/ice/migrate.sh

# Trigger a deploy from GitHub
# (Push to main, or: Actions tab → Deploy → Run workflow)

# Check disk usage
df -h
docker system df

# Clean up unused images
docker image prune -f

# Manual DB backup
/opt/ice/backup.sh

# Check Caddy certs
docker compose -f docker-compose.prod.yml exec caddy caddy list-certificates
```